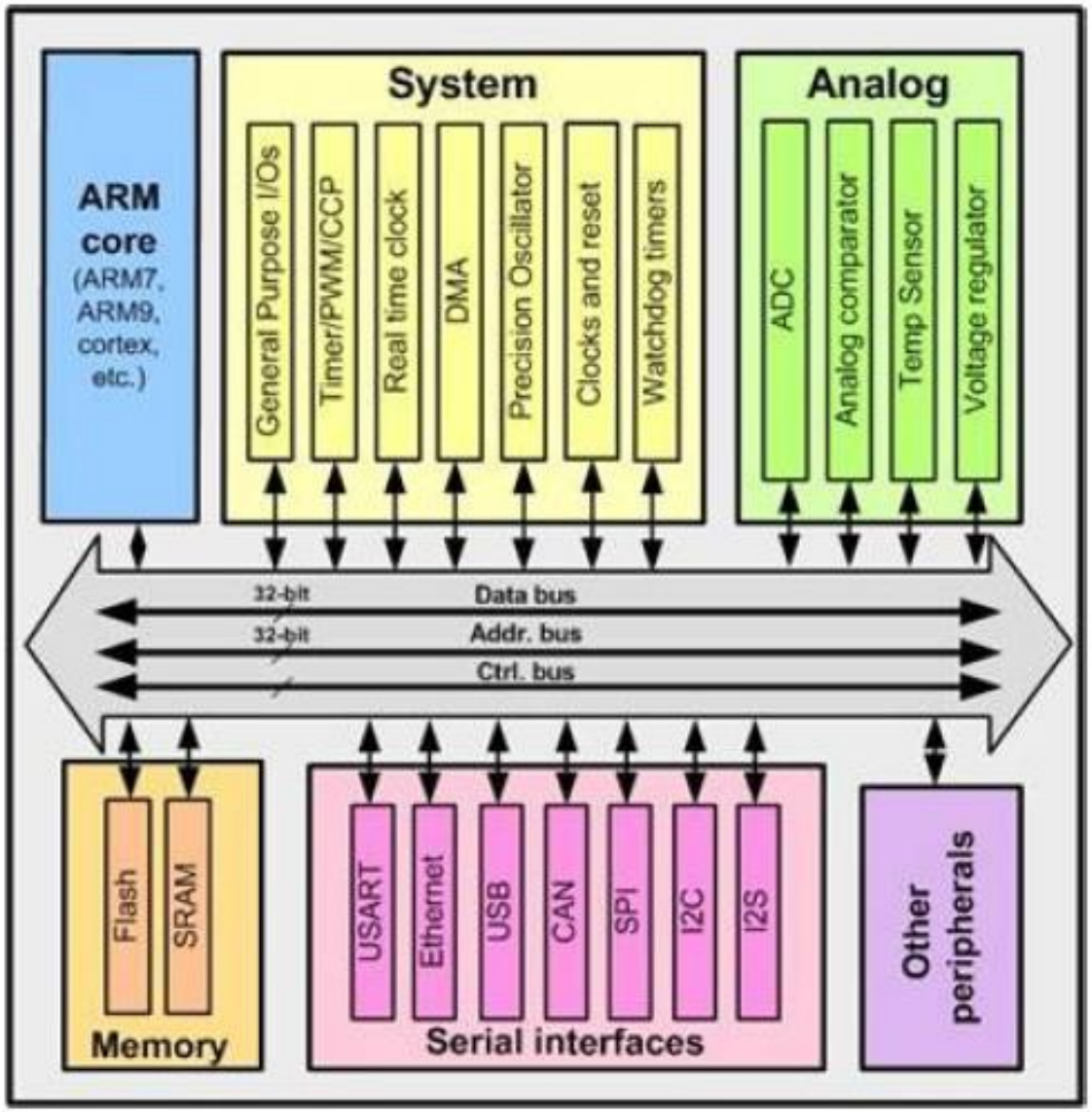
ARM Memory Map and Memory Access

The ARM CPU uses 32-bit addresses which gives us a maximum of 4 GB (gigabytes) of memory space. This 4GB of memory space has addresses 0x00000000 to 0xFFFFFFFF, meaning each byte is assigned a unique address (ARM is a byte-addressable CPU).

D31 D24	D23 D16	D15 D8	D7 D0	
0x00000003	0x00000002	0x00000001	0x00000000	0x00000000
0x00000007	0x00000006	0x00000005	0x00000004	0x00000004
0x0000000B	0x0000000A	0x00000009	0x00000008	0x00000008
0x0000000F	0x0000000E	0x0000000D	0x0000000C	0x0000000C
⋮	⋮	⋮	⋮	
0xFFFFFFF3	0xFFFFFFF2	0xFFFFFFF1	0xFFFFFFF0	0xFFFFFFF0
0xFFFFFFF7	0xFFFFFFF6	0xFFFFFFF5	0xFFFFFFF4	0xFFFFFFF4
0xFFFFFFFB	0xFFFFFFFA	0xFFFFFFF9	0xFFFFFFF8	0xFFFFFFF8
0xFFFFFFFF	0xFFFFFFFE	0xFFFFFFFD	0xFFFFFFFC	0xFFFFFFFC

ARM Assembly Language Programming & Architecture by Mazidi, et al.

ARM buses and memory access



Memory Connection Block Diagram in ARM

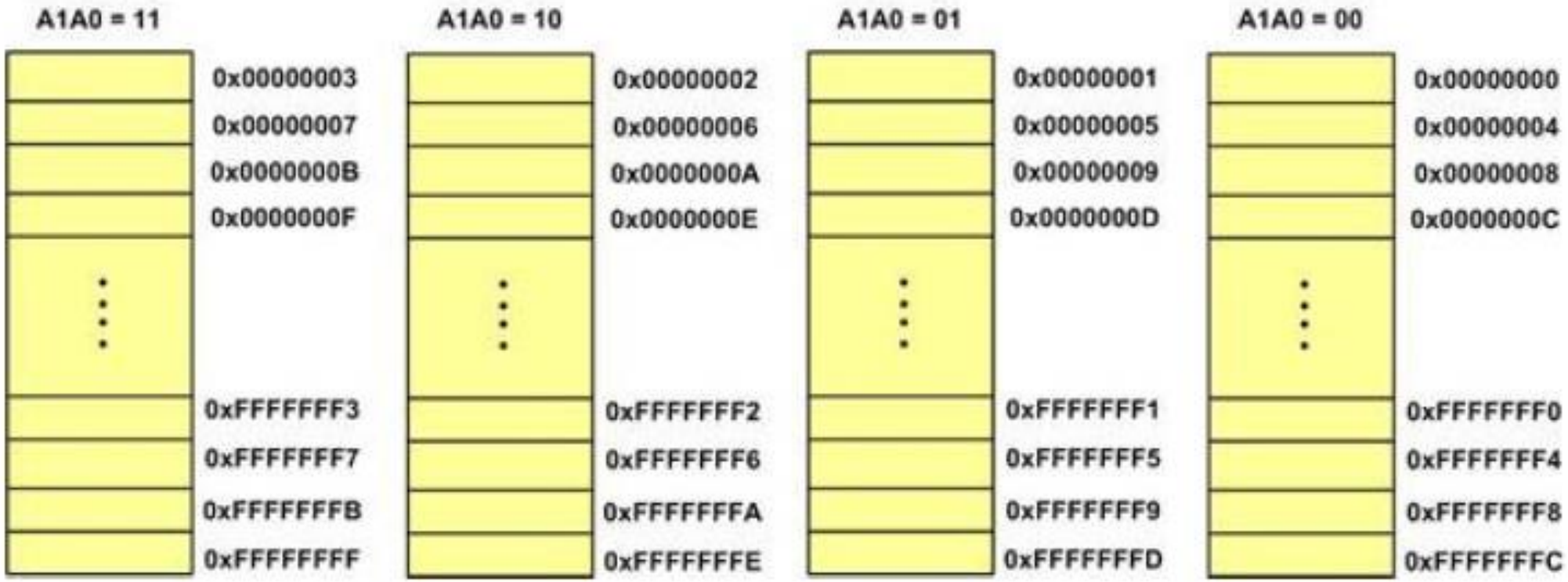
ARM Assembly Language Programming & Architecture by Mazidi, et al.

D31–D0 Data bus

The 32-bit data bus of the ARM provides the 32-bit data path to the on-chip and offchip memory and peripherals. They are grouped into 8-bit data chunks, D0–D7, D8–D15, D16–D23, and D24–D31.

A31–A0

These signals provide the 32-bit address path to the on-chip and off-chip memory and peripherals. Since the ARM supports data access of byte (8 bits), half word (16 bits), and word (32 bits), the buses must be able to access any of the 4 banks of memory connected to the 32-bit data bus. The A0 and A1 are used to select one of the 4 bytes of the D31-D0 data bus.

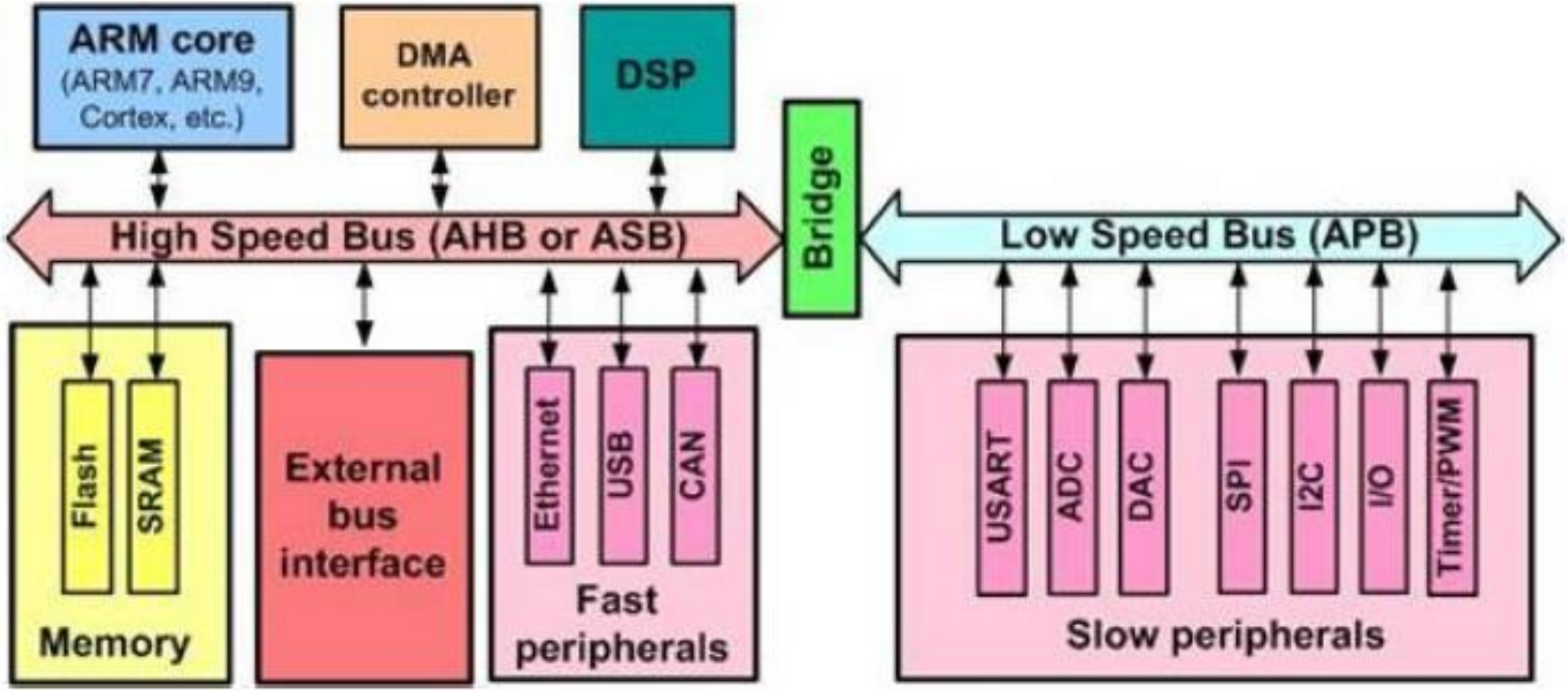


Memory Block Diagram in ARM

ARM Assembly Language Programming & Architecture by Mazidi, et al.

AHB and APB buses

The ARM CPU is connected to the on-chip memory via an AHB (advanced high performance bus). The AHB is used not only for connection to on-chip ROM and RAM, it is also used for connection to some of the high speed I/Os (input/output) such as GPIO (general purpose I/O). ARM chip also has the APB (advanced peripherals bus) bus dedicated for communication with the on-chip peripherals such as timers, ADC, serial COM, SPI, I2C, and other peripheral ports. While we need the 32-bit data bus between CPU and the memory (RAM and ROM), many slower peripherals are 8 or 16 bits and
there is no need for entire fast 32-bit data bus pathway. For this reason, ARM uses the AHB-to-APB bridge to access the slower on-chip devices such as peripherals.



AHB and APB in ARM

Bus cycle time

- To access a device such as memory or I/O, the CPU provides a fixed amount of time called a bus cycle time. The bus cycle time used for accessing memory is often referred to as MC (memory cycle) time. The time from when the CPU provides the addresses at its address pins to when the data is expected at its data pins is called memory read cycle time.

- While for on-chip memory the cycle time can be 1 clock, in the off-chip memory the cycle time is often 2 clocks.

Code memory region

- The 4 GB of ARM memory space is organized as 1G × 32 bits since the ARM instructions are 32-bit. The internal data bus of the ARM is 32-bit, allowing the transfer of one instruction into the CPU every clock cycle. The fetching of an instruction in every clock cycle can work only if the code is word aligned, meaning each instruction is placed at an address location ending with 0, 4, 8, or C.

SRAM memory region

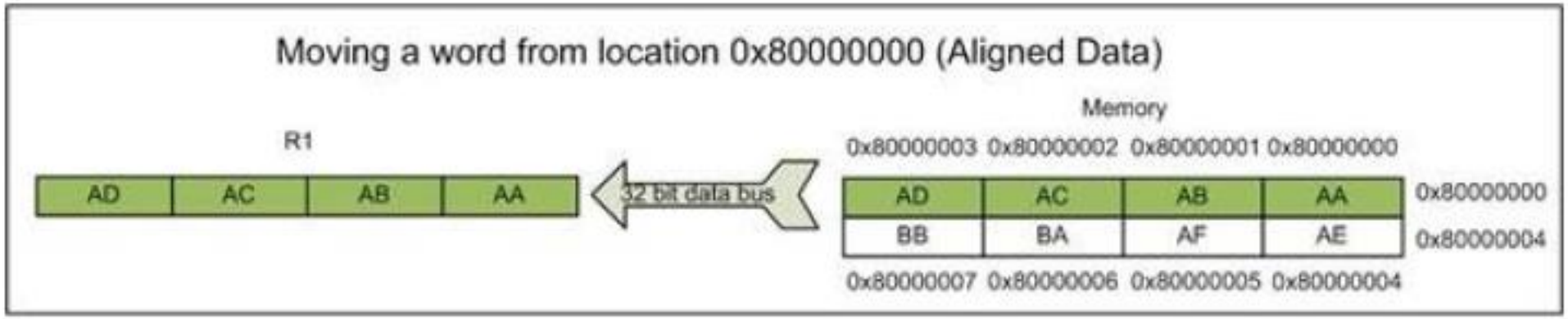
- A section of the memory space is used by SRAM. The SRAM can be on-chip or offchip (external). This on-chip SRAM is used by the CPU to store parameters. It is also used by the CPU for the purpose of the stack.

Data misalignment in SRAM

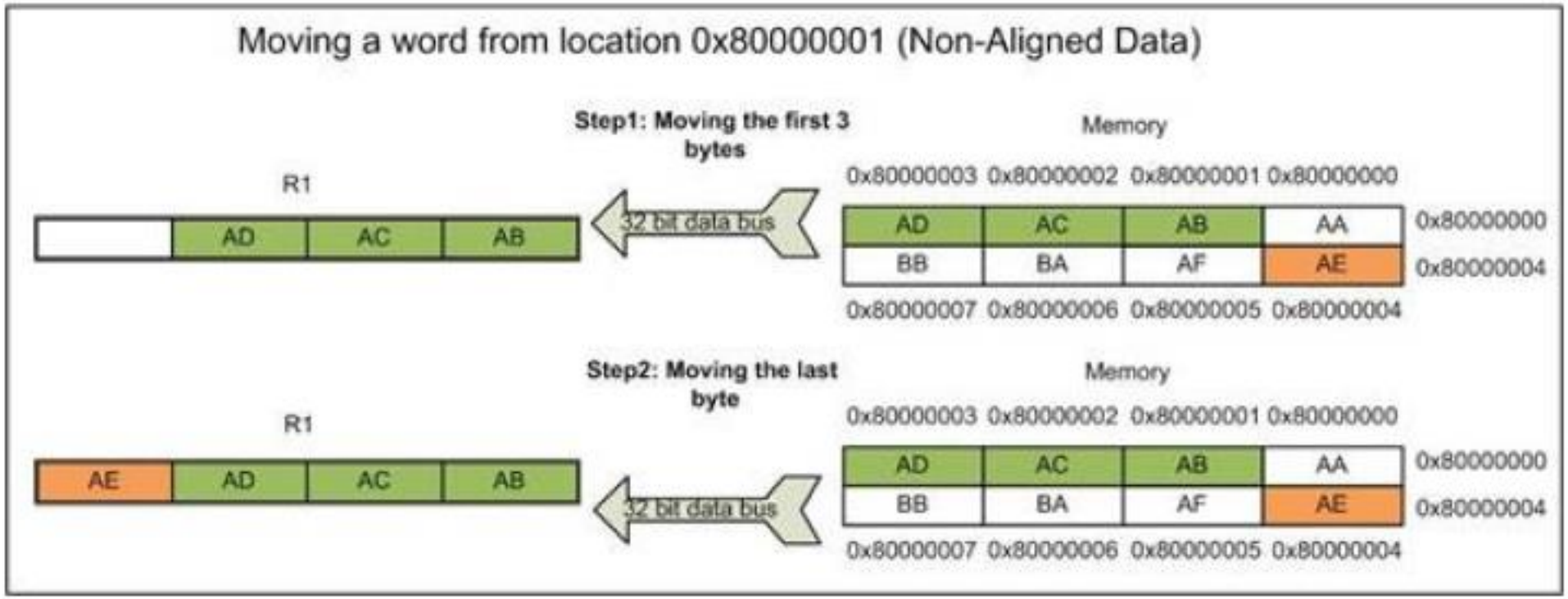
- If the data is aligned, for every memory read cycle, the ARM brings in 4 bytes of information (data or code) using the D31–D0 data bus.
- The single cycle access of memory is also used by ARM to bring into registers 4 bytes of data every clock cycle assuming that the data is aligned. To make sure that data are also aligned we use the align directive.

Accessing non-aligned data

LDR R1, [R0]
R0=0x80000000



LDR R1, [R0]
R0=0x80000001 8 bytes contents of memory locations 0x80000000 through 0x80000007 are being fetched in two consecutive Cycles.



Memory Access for Aligned and Non-aligned Data

ARM Assembly Language Programming & Architecture by Mazidi, et al.

Using LDR instruction with DCD and ALIGN directives

- The DCD and DCDU directives are used for 32-bit (word) data. The DCD directive ensures 32-bit data types are aligned, in contrast to DCDU which does not. DCD is used as follows:

  VALUE1 DCD 0x99775533
- This ensures that VALUE1, a word-sized operand, is located in a word aligned address location.
- An instruction accessing it will take only a single memory cycle.
- The one-time use of ALIGN directive at the beginning of data area using DCDU makes the data aligned for that group of data.

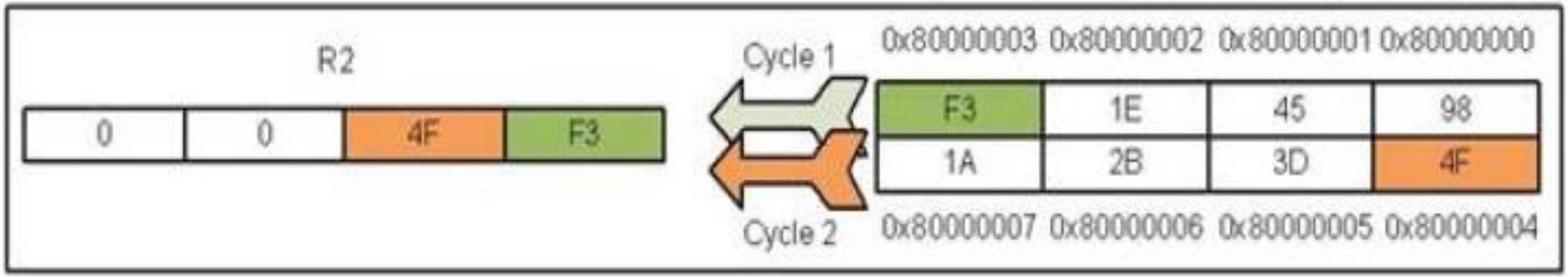
Using LDRH with DCW and ALIGN directives

- With DCWU, we must use the ALIGN directive in the data area of a given program to ensure they are aligned. This is in contrast to the DCW directive which ensures data type to be half-word aligned.
- This is especially the case when we use the LDRH instruction.

Example:

```
LDR R1,=0x80000000  ;R1=0x80000000
LDR R3,=0xF31E4598  ;R3=0xF31E4598
LDR R4,=0x1A2B3D4F ;R4=0x1A2B3D4F
STR R3,[R1]
STR R4,[R1,#4]
LDRH R2, [R1]
LDRH R2, [R1,#1]
LDRH R2, [R1,#2]
LDRH R2, [R1,#3]
```

LDRH R2, [R1]
LDRH R2, [R1,#1]
LDRH R2, [R1,#2] instructions take only one memory cycle.
But LDRH R2, [R1,#3] will take two memory cycles to load the data



Using LDRB with DCB and ALIGN directives

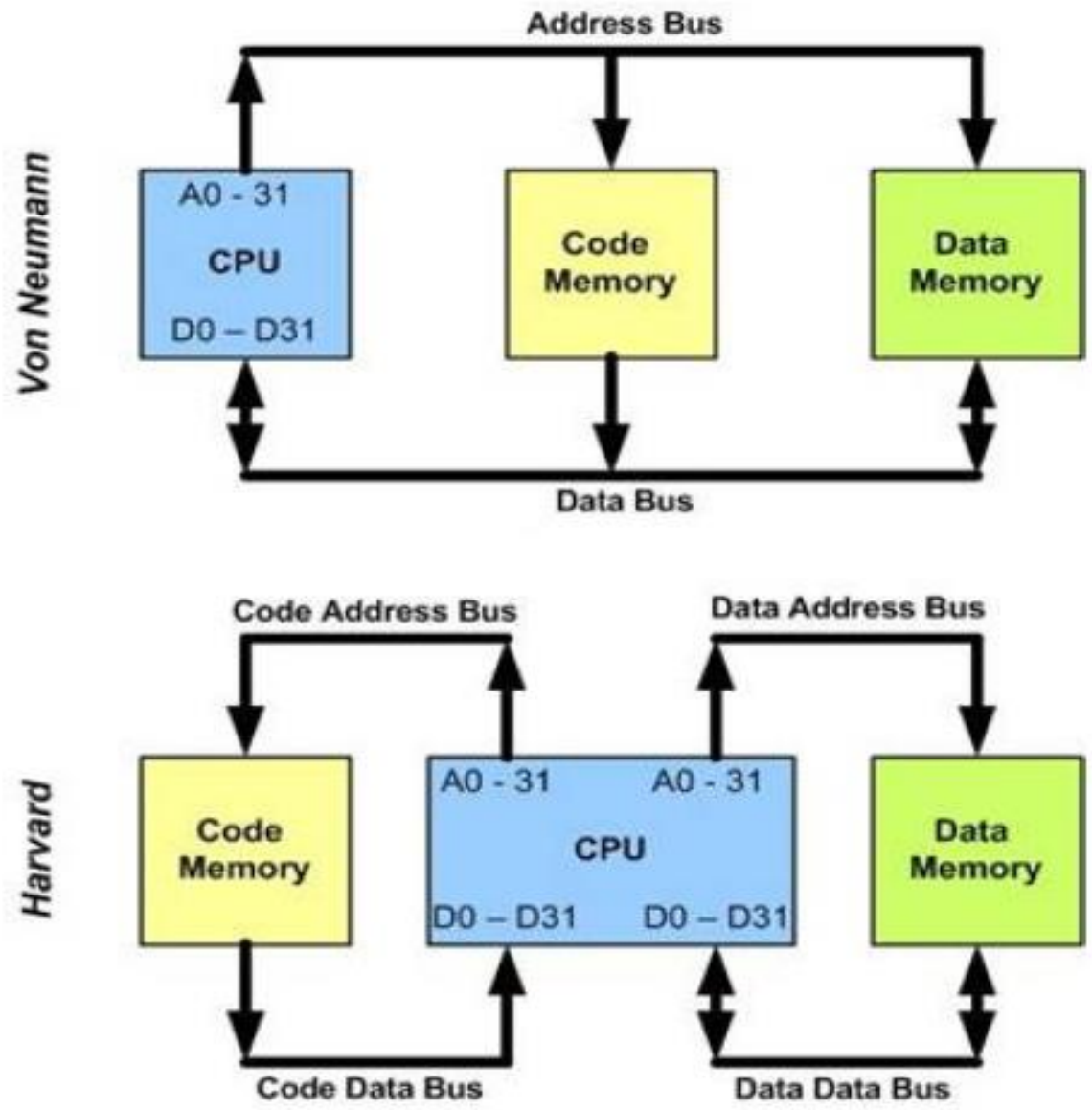
- The problem of misaligned data does not exist when the data size is bytes. In cases such as using the string of ASCII characters with the DCB directive, accessing a byte takes the same amount of time (one memory cycle) as an aligned word (4 bytes), regardless of the address location of the data.

Peripheral region

A section of memory is set aside for peripherals. The type of peripherals and memory address locations used is unique to a vendor. The ARM manufacturers provide the details of memory map for the peripherals.

Harvard Architecture and ARM

- In recent years many ARM manufacturers are using the Harvard architecture for ARM CPUs.
- Old ARM architectures up to ARM7 use Von Neumann architecture.
- The Harvard architecture feeds the CPU with both code and data at the same time via two sets of buses, one for code and one for data. This increases the processing power of the CPU since it can bring in more information.



Von Neumann vs. Harvard Architecture

ARM Assembly Language Programming & Architecture by Mazidi, et al.

Stack and Stack Usage in ARM

Nested calls in ARM:

- Upon calling a subroutine from the main program using the BL instruction, R14, the linker register, keeps track of where the CPU should return after completing the subroutine. Now, if we have another call inside the subroutine using the BL instruction, then it is our job to store the original R14 on the stack. Failure to do that will end up crashing the program.
- The stack is a section of RAM used by the CPU to store information temporarily.
- This information could be data or an address or CPU registers when calling a subroutine.
- Stack is also widely used when executing an interrupt service routine (ISR).

How stacks are accessed

- If the stack is a section of RAM, there must be a register inside the CPU to point to it. In the ARM CPU the register used to access the stack is R13.
- The storing of CPU information such as the registers on the stack is called a PUSH, and loading the contents of the stack back into a CPU register is called a POP. In other words, a register is pushed onto the stack to save it and popped off the stack to retrieve it.

Pushing onto the stack

- The stack pointer (SP) points to the top of the stack (TOS).
- As we push (store) data onto the stack, the data are saved in RAM (where the SP points to) and SP must be decremented (or incremented) to point to the next location.
- In the ARM we have a choice of either incrementing the SP or decrementing it. We must actually code the instruction for stack pointer decrementation (or incrementation)
- To push a register onto stack we use the STR and SUB instructions

```
STR Rr, [R13]          ;Rr can be any registers (R0-R12)
SUB R13, R13, #4       ;decrement stack pointer
```

For example, to store the value of R1 we can write the following instructions:

```
STR R1, [R13]          ;store R1 onto the stack,
SUB R13, R13, #4       ;decrement SP
```

Popping from the stack

- Popping (loading) the contents of the stack back into a given register is the opposite process of pushing. When the POP is executed, the SP is incremented (or decremented) and the top location of the stack is copied (loaded) back to the register.

- The stack is LIFO (Last-In-First-Out) memory.

- To pop into a register from the stack we use the LDR and ADD instructions

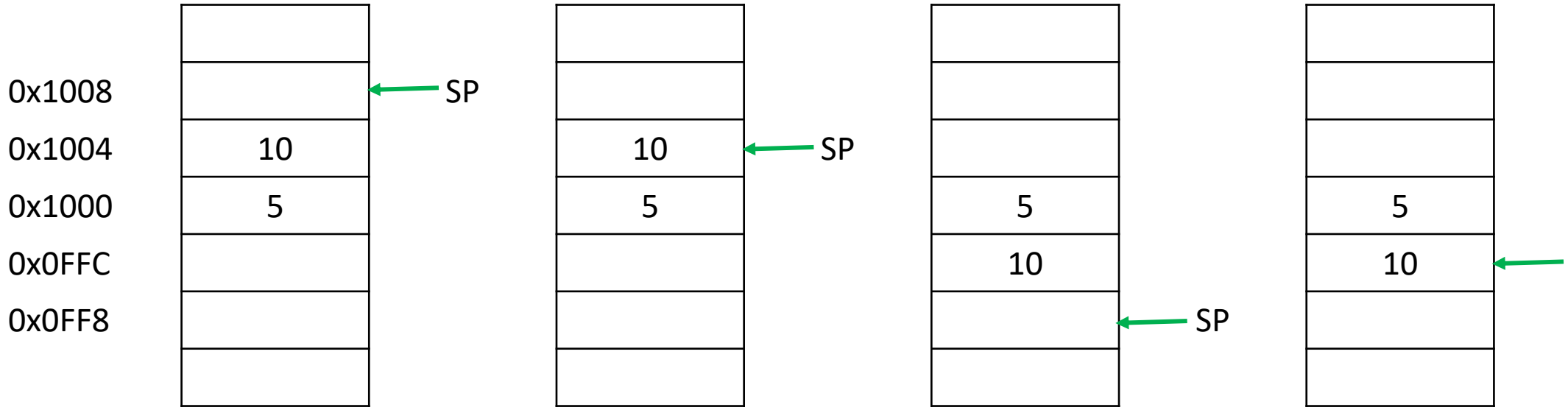
To retrieve data from stack we can use the LDR instruction.

```
ADD R13, R13, #4   ;increment stack pointer
LDR Rr, [R13]      ;Rr can any of the registers (R0-R13)
```

For example, the following instructions pop from the top of stack and copy to R1:

```
ADD R13, R13, #4   ;increment SP
LDR R1, [R13]      ;load (POP) the top of stack to R1
```

- We have four stack structure, either it is ascending or descending. The stack is called ascending when it is incremented after each store (PUSH) instruction and decremented after each load (POP) instruction.
- It is called descending when it is decremented after each store (PUSH) instruction and incremented after each load (POP) instruction.
- The stack pointer can point to the last filled location; in this case the stack is called Full Stack.
- The stack pointer can point to the next available location, as well; which is called an Empty Stack.



	Empty Ascending		Fully Ascending		Empty Descending		Fully Descending
	PUSH		**PUSH**		**PUSH**		**PUSH**
STMIA	STR R0, [R13]	STMIB	ADD R13, #4	STMDA	STR R0, [R13]	STMDB	SUB R13, #4
	ADD R13, #4		STR R0, [R13]		SUB R13, #4		STR R0, [R13]
	POP		**POP**		**POP**		**POP**
LDMDB	SUB R13, #4	LDMDA	LDR R0, [R13]	LDMIB	ADD R13, #4	LDMIA	LDR R0, [R13]
	LDR R0, [R13]		SUB R13, #4		LDR R0, [R13]		ADD R13, #4

Using LDM and STM instructions for the stack

Another way to push register contents onto the stack is to use STM (store multiple) and LDM (load multiple) instructions.

STM

```
STM R11, {R0-R3}
;Store R0 through R3 onto memory pointed to by R11

STM R7, {R0,R3,R5}
;Store R0, R3, R5 onto memory pointed to by R7
```

LDM

```
LDM R11, {R0-R3}
;Load R0 through R3 from memory pointed to by R11

LDM R7, {R0,R3,R5}
;Load R0,R3,R5 from memory pointed to by R7
```

- To support 4 types of the stack STM and LDM instructions take four suffixes IA, IB, DA and DB.
- If no suffix is used, the default action is IA

Writeback options of STM and LDM

We can specify the action to be taken for the pointer. The action can be increment or decrement before or after the push or pop is done.

IA Increment After

IA stands for Increment After and adds four to the pointer after load or storing each register.

IB Increment Before

IB stands for Increment Before and adds four to the pointer before load or storing each register.

DA Decrement After

DA stands for Decrement After and subtracts four from the pointer after load or storing each register.

DB Decrement Before

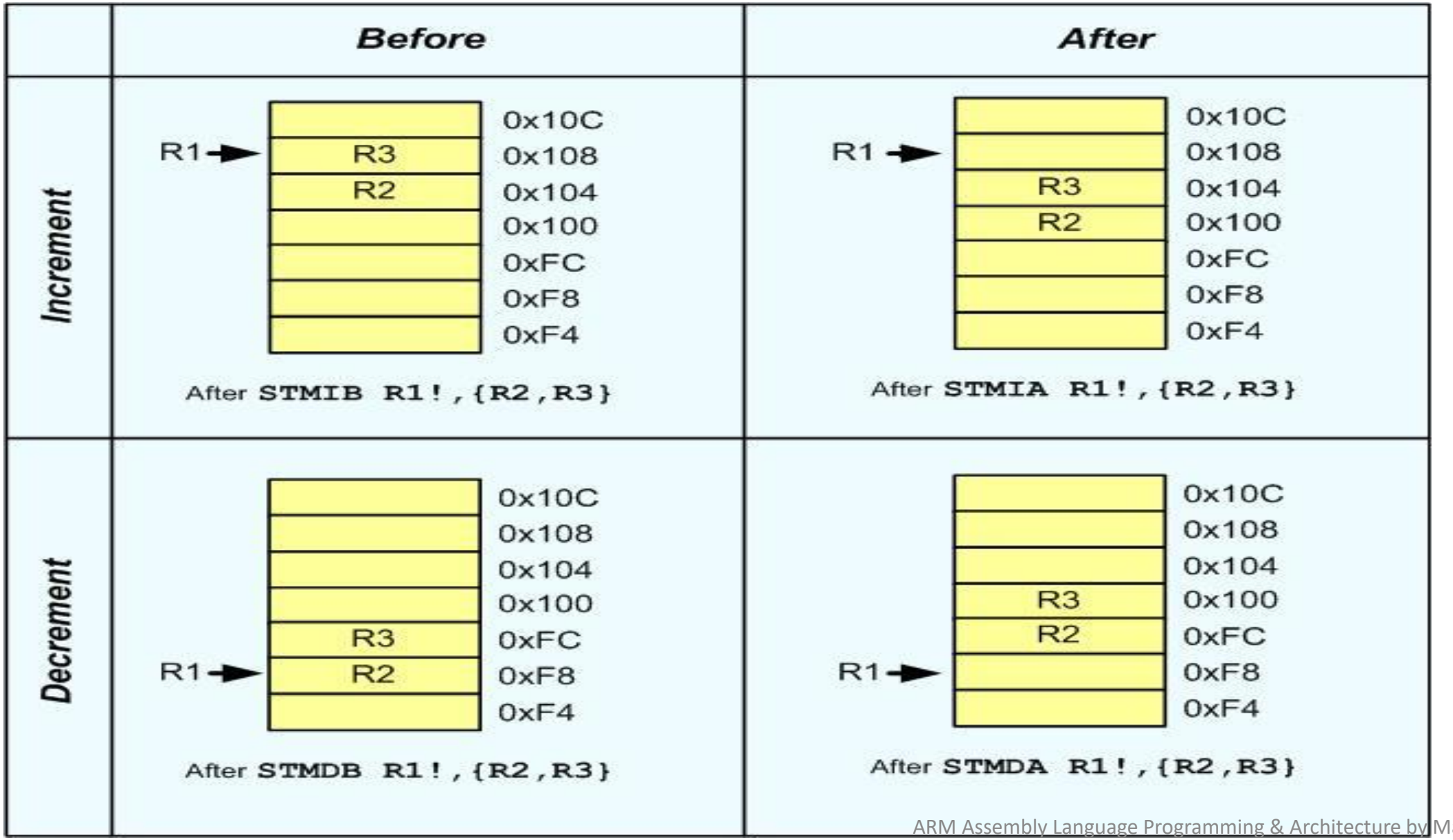
DB stands for Decrement Before and subtracts four from the pointer before load or storing each register.

Stack Structure	Load	Store	Load (alternate Names)	Store (alternate Names)
Full Ascending	LDMDA	STMIB	LDMFA	STMFA
Full Descending	LDMIA	STMDB	LDMFD	STMFD
Empty Ascending	LDMDB	STMIA	LDMEA	STMEA
Empty Descending	LDMIB	STMDA	LDMED	STMED

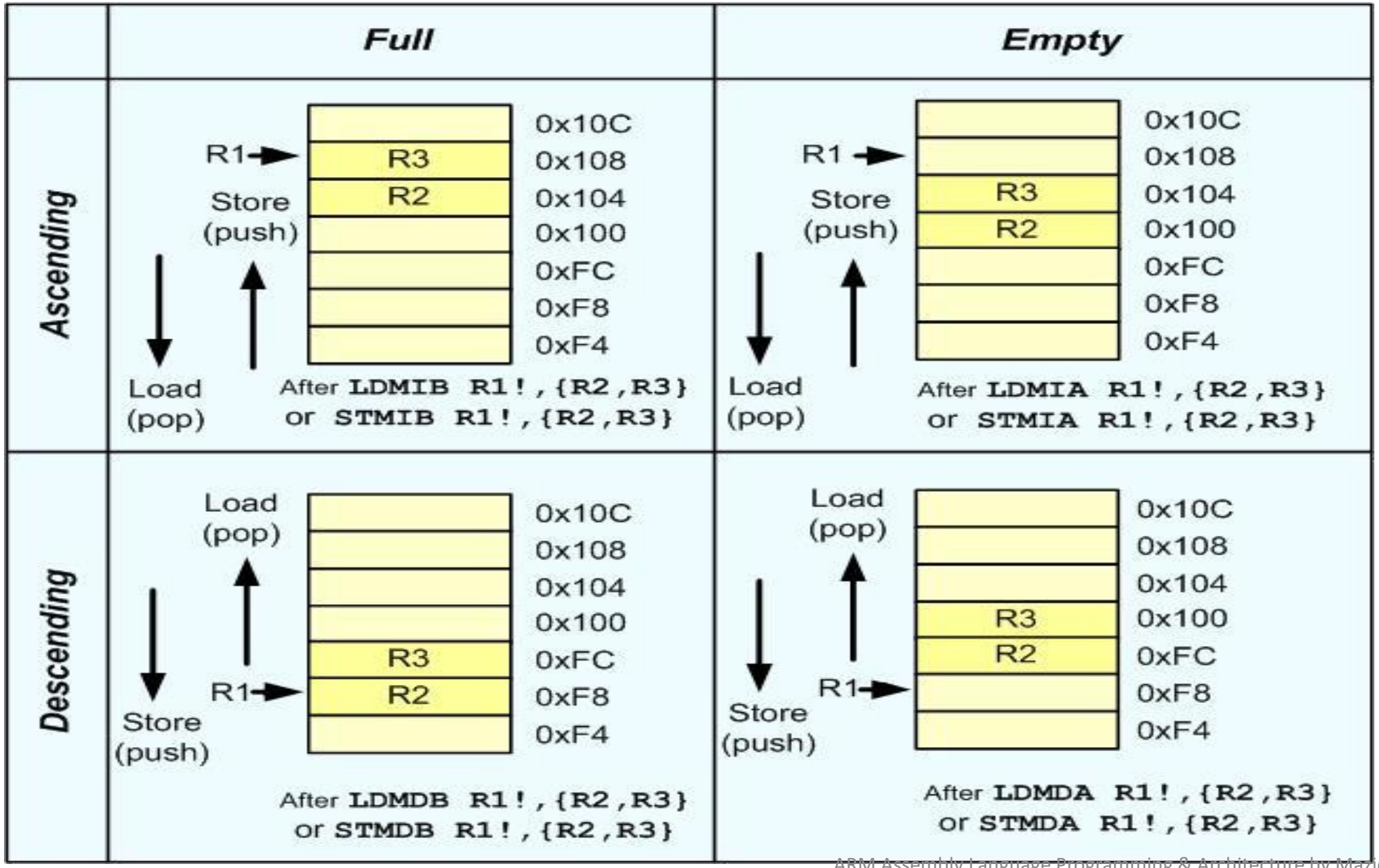
The four stack structures and the options of LDM and STM instructions

The stack structure used by ARM for PUSH, POP instructions and interrupt handling is a Full Descending stack using R13 as the stack pointer. To support Full Descending stack, STMDB and LDMIA pair of instructions should be used.

For further clarification assume that R1 = 0x100. Figure below shows the memory after running STM R1!,{R2,R3} with each of IA, IB, DA and DB options.



ARM Assembly Language Programming & Architecture by Mazidi, et al.


Full
Empty
Ascending
Descending
R1
R3
R2
Store
(push)
Load
(pop)
0x10C
0x108
0x104
0x100
0xFC
0xF8
0xF4
After LDMIB R1!,{R2,R3}
or STMIB R1!,{R2,R3}
After LDMIA R1!,{R2,R3}
or STMIA R1!,{R2,R3}
After LDMDB R1!,{R2,R3}
or STMDB R1!,{R2,R3}
After LDMDA R1!,{R2,R3}
or STMDA R1!,{R2,R3}

STMDB Store Multiple register and Decrement Before

Flags: Unaffected.

Format: STMDB Rn,{Rx,Ry,…}

Function: Stores registers Rx, Ry,… into consecutive memory locations. The starting address of memory location is given by Rn register. The source registers are separated by comma and placed in braces.

```
STMDB R11!, {R0, R1, R2, R3}
STMDB R11!, {R0-R3}
;Store R0 through R3 onto memory pointed to by R11 and update R11
with the final address

STMDB R11!, {R0, R5, R3}
;Store R0, R3 and R5 onto memory pointed to by R11 and update R11
with the final address
```

STMDB R11!, {R0-R3, R8, R7}
;Store R0 through R3, R7 and R8 onto memory pointed to by R11 and update R11 with the final address

LDMIA Load Multiple registers and Increment after each Access
Flags: Unaffected.
Format: LDMIA Rn, {Rx,Ry,..}
Function: This is the same as the LDM instructions. This IA (Increment the address after each Access) is the default. We use this for Poping (loading) multiple words from descending stack into CPU registers.

LDMIA R11!, {R0, R1, R2, R3}
LDMIA R11!, {R0-R3}
LDMIA R11!, {R0, R5, R3}

When the registers are pushed on to the stack, lower numbered registers are stored in the lower address and when popped, data from lower address goes into the lower numbered register.

PUSH and POP instructions.

PUSH PUSH register onto stack

Flags: Unaffected.

Format: PUSH {reg_list} ;PUSH reg_list onto stack

Function: Copies the contents of registers stated in reg_list onto the stack and decrements SP by 4, 8, 12, 16, … depending on the number of registers in reg_list.

Example:

PUSH {R1} ;PUSH the R1 onto top of stack

PUSH {R1,R4,R7} ;PUSH R1,R4,R7 onto top of stack

PUSH {R2-R6} ;PUSH the R2,R3,R4,R5,R6 onto top of stack

PUSH {R0,R5} ;PUSH the R0 and R5 onto top of stack

PUSH {R0-R7} ;PUSH the R0 through R7 onto top of stack

The PUSH instruction is alias for STMDB R13!.

POP POP register from Stack

Flags: Unaffected.

Format: POP {reg_list} ;reg_reg = words off top of stack

Function: Copies the words pointed to by the stack pointer to the registers indicated by the reg_list and increments the SP by 4, 8, 12, 16, … depending on the number of registers in the reg_list.

Example:

POP {R1} ;POP the top word of stack to R1

POP {R1,R4,R7} ;POP the top 3 words of stack to R1,R4,R7

POP {R2-R6} ;POP the top 5 words of stack to R2-R6

POP {R0,R5} ;POP the top 2 words of stack to R0 and R5

POP {R0-R7} ;POP the top 8 words of stack to R0-R7

The POP instruction is alias for LDMIA R13!.